Lab 2.b demo 2 walkthrough — the git hook (layer 3, commit time)

The instructor's git-hook demo, written down so you can run it in **your own copy**. In the live session only the instructor did this — it's the "hook you already know" that the Claude hook is then taught from. Doing it yourself is a stretch goal, not an acceptance criterion. ~10 minutes.

The rule it enforces already exists. `CLAUDE.md` in this repo says *"Conventional Commits"* — and nothing anywhere checks it. That's the same gap the whole lab is about (a written rule with no gate), one layer down the timeline: this hook holds against **anyone committing locally**, agent or human, Claude Code open or not.

Before you start: your own copy, any branch, `npm install` already run.

Platforms: every step is identical on macOS, Linux, and Windows — PowerShell aliases `cp` and `rm`, `npm` / `npx` / `git` are the same everywhere, and Git Bash takes the commands unchanged. The steps here match the slide's rows 1:1, and step 3 explains *why* there is nothing to adapt: git never asks your operating system to execute the hook file.

Step 1 — Install husky

All platforms:

```
npm install -D husky
npx husky init
```

You should see: a `.husky/` folder appears, and `package.json` gains `"prepare": "husky"` — that's what wires the folder up for everyone who runs `npm install` later.

`husky init` also writes a starter `.husky/pre-commit` containing `npm test` — **delete it for this demo** (same command in every shell):

```
rm .husky/pre-commit
```

Why this matters: git runs `pre-commit` **before** `commit-msg`. If the starter hook fails, your hook never runs — and in a repo with no `test` script it fails on *every* commit with a confusing, unrelated `npm error Missing script: "test"`. (This repo's `npm test` exists and is green, so here it would merely slow each commit; in a real repo without one, it blocks everything.)

Step 2 — Copy the hook in

The same command on every platform (PowerShell aliases `cp`):

All platforms:

```
cp docs/labs/snippets/commit-msg.sh .husky/commit-msg
```

What you just copied — the whole check is five working lines:

```
msg="$(head -n 1 "$1")"

if echo "$msg" | grep -qE '^(feat|fix|docs|test|refactor|chore|build|ci)(\([a-z0-9-]+\))?: .+'; then
  exit 0
fi

echo 'BLOCKED: commit message is not Conventional Commits.' >&2
# ...Why / Instead / Done means — the same four-part message shape as the Claude hooks
exit 1
```

Note the anatomy — you'll meet it again unchanged at authoring time in demo 3: **an event fires a script, and the exit code decides.** Here the event is your commit and the input is the message file git passes as `$1`; there the event is the agent's tool call and the input is JSON on stdin.

Step 3 — Why this runs everywhere (nothing to run)

Notice what we did **not** do: no `chmod`, no `.sh` / `.ps1` extension, and the shebang line is decorative. That's because the file is never "executed" by your operating system at all:

1. `npx husky init` set `git config core.hooksPath` to `.husky/_` — a folder of **shims** husky generates. Git runs the shim, not your file.
2. The shim runs your file **as an argument to the shell**: `sh -e .husky/commit-msg`. A file passed to `sh` needs no executable bit and no extension — on any OS.
3. On Windows, hooks never touch the `.ps1` / `.bat` extension rules: `git.exe` finds and spawns hooks itself, and Git for Windows ships its own `sh` (plus `grep`, `head`, …) for exactly this. Same file, same behavior.

You can verify claim 2 yourself: strip the bit (`chmod -x .husky/commit-msg`) and the hook still fires.

The one thing the shell **does** care about: **line endings.** A hook saved with Windows CRLF endings fails *closed* — every commit, valid ones included, dies with `: command not found` (code 127) or a syntax error. Save hook files with LF (VS Code: click "CRLF" in the status bar → LF). This repo's `.gitattributes` keeps the snippet LF at checkout, but a Windows editor can reintroduce CRLF when you edit.

> **The caveat that keeps this honest:** *raw* git hooks — files in `.git/hooks/`, or a `core.hooksPath` aimed directly at your scripts, no husky — ARE executed directly by git. There, `chmod +x` is required on macOS/Linux, and the bit reaches teammates through git's index: `git update-index --chmod=+x <hook>`. Different plumbing, same anatomy.

Step 4 — Verify the BLOCK

```
git commit --allow-empty -m "changed stuff"
```

Refused — the BLOCKED/Why/Instead/Done message prints, the exit code is non-zero, and the commit **never happens** (`git log --oneline -1` is unchanged). `--allow-empty` just means you don't need staged changes to test.

Step 5 — Verify the ALLOW

```
git commit --allow-empty -m "docs: explain the commit format"
```

Passes, silently — exit 0 is the 99% case. A hook that fires on legitimate work gets disabled by Friday and takes the layer with it, so this half of the verification is not optional.

Step 6 — Meet the bypass

```
git commit --no-verify -m "changed stuff"
```

It lands. Every layer has a bypass and this one **ships with git** — you can't remove it, so write it down (a line in `CLAUDE.md` next to the rule, naming when `--no-verify` is acceptable and who approves). An undocumented bypass is the real governance hole, not the missing gate.

Clean up the two demo commits (in your terminal — this is your own branch):

```
git reset --hard HEAD~2
```

Step 7 — The honest caveat (nothing to run)

Clone the repo fresh somewhere and commit **without** running `npm install`: no hooks. Layer 3 travels with the wired clone, not with the repo — and `--no-verify` skips it at will. Both are exactly why the same rule runs **again at merge time**, where neither escape exists. That's demo 4 and the gate you build in the exercise.

Same anatomy, three moments in time

	Git hook (commit)	Claude hook (authoring)	CI gate (merge)
Fires when	you commit	the agent acts — a tool call	a PR opens or updates
Input	the commit-message file (`$1`)	the tool input, JSON on stdin	the diff vs the base branch
Verdict	non-zero exit → no commit	exit 2 → blocked; stderr → agent	failed check → no merge
Holds against	anyone committing, wired clone	the agent, at authoring time	anyone merging — agent or human
The bypass	`git commit --no-verify`	edit `settings.json`	the documented escape hatch

Common wobbles

Wobble	The move
Hook never fires	`npm install` run since husky was added? `"prepare": "husky"` in package.json? `git config core.hooksPath` says `.husky/_` ?
Every commit fails: `npm error Missing script: "test"`	Husky's starter `.husky/pre-commit` (`npm test`), not your hook — pre-commit runs **before** commit-msg. Delete it, or add a test script
Every commit blocked: `: command not found` (code 127) or a syntax error	The hook file has Windows CRLF line endings — save as LF (VS Code: status-bar CRLF → LF). `core.autocrlf=true` can also convert files at checkout
Writing a RAW git hook later (no husky)	There the exec bit is real: `chmod +x` locally, and it reaches teammates through git's index — `git update-index --chmod=+x <hook>`
Blocked on a message you think is fine	Read the message — the regex IS the policy. If your team needs another type (e.g. `perf`), extend the list **on purpose**, in a PR
Works for you, not for a teammate	They haven't run `npm install` since the hook landed — layer 3 is per-clone
Your real repo already runs husky	Add `commit-msg` as a new file next to the existing `pre-commit` — hooks are per-event files; they compose, you never replace
Tempted to grow the regex forever	That's the graduation signal: `commitlint` is the grown-up version of these five lines. Five lines teach the anatomy; deps handle scale